
Informe de Ambientes

Proyecto: Autogestión-Sena

ÍNDICE

1. Introducción
2. Objetivos del documento
3. Estructura de Ambientes en GitHub
 - 3.1. ¿Qué es un ambiente en control de versiones?
 - 3.2. Ambientes utilizados en Autogestión SENA
4. Flujo de trabajo (Workflow) por ambientes
 - 4.1. Ambiente *dev*
 - 4.2. Ambiente *qa*
 - 4.3. Ambiente *staging*
 - 4.4. Ambiente *main*
5. Uso de ambientes por repositorio
 - 5.1. Frontend Web (React + TS + Vite)
 - 5.2. Backend (Django REST Framework)
 - 5.3. Aplicación móvil (.NET MAUI)
6. Problemas encontrados y soluciones implementadas
7. Ejemplos con capturas (Frontend)
8. Uso de Cherry Pick y Fork
9. Conclusiones

1. Introducción

El presente documento describe detalladamente la estrategia de manejo de ambientes utilizada en el proyecto Autogestión SENA dentro de la plataforma GitHub.

El propósito fundamental de esta estructura de ambientes es garantizar un flujo de desarrollo organizado, controlado y progresivo, permitiendo que cada funcionalidad pase por etapas de verificación, pruebas y despliegue antes de llegar a producción.

La gestión de ambientes es un componente esencial en proyectos de mediana y gran escala, ya que permite asegurar la calidad del software, facilitar el trabajo en equipo, evitar conflictos entre ramas y reducir riesgos durante las implementaciones.

En el caso de Autogestión SENA, se adoptó un flujo basado en GitFlow adaptado, utilizando las ramas:

- dev → desarrollo activo
- qa → pruebas funcionales
- staging → pre-producción
- main → versión estable o producción

Estas etapas, combinadas con ramas específicas por cada Historia de Usuario (HU), permitieron mantener un control más granular del progreso.

Este informe detalla cómo se utilizó cada ambiente, cómo se manejaron las HU, el proceso de merge request, el uso de cherry pick para resolver inconsistencias y cómo se aplicó esta metodología en los tres repositorios principales: frontend web, backend y móvil.

En particular, se resalta que el frontend fue el proyecto donde se logró aplicar completamente el sistema de ambientes, mientras que en backend y móvil se presentaron dificultades técnicas que limitaron la implementación completa, pero aun así se documenta el proceso para fines académicos y de aprendizaje.

2. Objetivos del documento

- Objetivo general:
 - Describir de manera completa el proceso de manejo de ambientes realizado en GitHub para el proyecto Autogestión SENA.
- Objetivos específicos:
 - Explicar la función de cada ambiente (dev, qa, staging, main).
 - Documentar el flujo de trabajo por cada Historia de Usuario.
 - Registrar el uso de ramas temporales por HU.
 - Detallar el proceso de integración (MR) entre ambientes.
 - Evidenciar el uso de cherry pick para resolver conflictos o trasladar cambios específicos.
 - Presentar ejemplos reales ejecutados durante el proyecto (HU-42 y HU-43).
 - Dejar registro del funcionamiento exitoso en frontend y de las dificultades en backend y móvil.

3. Estructura de Ambientes en GitHub

3.1. ¿Qué es un ambiente en control de versiones?

Un ambiente es un espacio lógico que representa un **estado del software** dentro del ciclo de vida.

Cada ambiente permite evaluar el proyecto en diferentes etapas:

- **Desarrollo**
- **Pruebas**
- **Preproducción**
- **Producción**

Esto evita mezclar código experimental con código estable y garantiza calidad en los despliegues.

3.2. Ambientes utilizados en Autogestión SENA

El proyecto utiliza un modelo basado en **GitFlow**, en el que cada ambiente representa una fase de maduración del software:

Ambiente	Propósito
dev	Desarrollo diario, nuevas funcionalidades y correcciones.
qa	Pruebas funcionales y validaciones internas.
staging	Simulación de producción antes del lanzamiento.
main	Versión estable y lista para despliegue final.

Cada Historia de Usuario (HU) crea **una rama por ambiente**, con el patrón:

- HU-XX-dev
- HU-XX-qa
- HU-XX-staging

Solo en el ambiente **main** se utiliza una rama especial:

- release.X.X

4. Flujo de trabajo (Workflow) por ambientes

A continuación se documenta el flujo exacto utilizado para cada ambiente, basado en ramas por HU (Historia de Usuario).

4.1. Ambiente *dev*

Es donde inicia el desarrollo de cada HU.

Es un ambiente dinámico y cambiante.

Procedimiento:

```
git pull origin dev
git switch dev
git switch -c HU-01-dev
# → desarrollo
# → commit y push
Crear MR desde: HU-01-dev → dev
```

Se cierra el desarrollo cuando la rama **dev** queda actualizada con la HU aprobada.

Acciones:

- Desarrollo de la funcionalidad.
- Corrección de errores iniciales.
- Integración con otras HU.

Se finaliza con un **merge request (MR)** hacia *dev*.

4.2. Ambiente *qa*

Para pruebas funcionales y validaciones.

Procedimiento:

```
git pull origin qa
git switch qa
git switch -c HU-01-qa
# → desarrollo QA
Crear MR: HU-01-qa → qa
```

Se cierra una vez la HU se aprueba en las pruebas.

Acciones:

- Verificación funcional.
- Validación de requisitos.
- Revisión por parte del equipo.

MR hacia *qa*.

4.3. Ambiente *staging*

Antes de llevar a main, staging permite probar el sistema completo.

Procedimiento:

```
git pull origin staging
git switch staging
git switch -c HU-01-staging
# → ajustes finales
Crear MR: HU-01-staging → staging
```

Esta rama simula el entorno real antes de producción.

Acciones:

- Pruebas finales.
- Validación previa al release.
- Revisión del comportamiento real del sistema.

MR hacia staging.

4.4. Ambiente main

Es la rama más importante: código listo para publicación.

Procedimiento:

```
git pull origin main  
git switch main  
git switch -c release.1.1
```

Dentro de esa release ingresan todas las HU del sprint:

```
HU-01-release.1.1  
HU-02-release.1.1  
HU-03-release.1.1  
...
```

Luego:

Crear MR: release.1.1 → main

Nunca se debe trabajar directamente sobre main.

Flujo de trabajo por HU:

1. Crear rama en **dev**
2. Desarrollar HU
3. Merge HU → dev
4. Crear rama HU en **qa**
5. Pruebas
6. Merge HU → qa
7. Crear rama HU en **staging**
8. Pruebas pre-release
9. Merge HU → staging
10. Crear rama release
11. Merge release → main

5. Uso de ambientes por repositorio

5.1. Frontend Web (React + TS + Vite)

Este repositorio fue el que sí logró implementar exitosamente todo el manejo de ambientes.

- Se trabajaron ramas dev, qa, staging y main.
- Se crearon ramas por HU por cada ambiente.
- Se hicieron merges controlados.
- Mantiene commits limpios y un flujo ordenado.

5.2. Backend (Django REST Framework)

En el backend se intentó implementar el mismo flujo, pero no se logró completamente.

Razones:

- Dependencias rotas entre ramas.
- Migraciones de base de datos generaban conflictos.
- Diferencias entre entornos locales impedían mover código fácilmente.
- Se trabajó más directo sobre dev y main.

Resultado: el manejo de ambientes no fue tan sólido como en frontend.

5.3. Aplicación Móvil (.NET MAUI)

También se intentó el manejo de ambientes, pero surgieron fallos.

- La UI cambiaba constantemente.
- Problemas de sincronización entre repositorios.
- Se avanzó más lento debido al enfoque visual.

El flujo se implementó parcialmente.

6. Problemas encontrados y soluciones implementadas

Problema	Causa	Solución
Conflictos entre ramas	Migraciones y cambios simultáneos	Establecer un orden de commits y revisar antes de merge
Fallo en control de versiones del móvil	Cambios rápidos	Acordar estándares de commits
CI fallaba por dependencias	Versiones diferentes	Unificar versiones en equipo
Muchas ramas abiertas	Falta de cierre	Políticas de merge estrictas

7. Ejemplos con capturas (Frontend)

Aclaracion:

Durante el desarrollo se implementó el uso de cherry-pick para trasladar cambios específicos entre ramas y asegurar la correcta gestión de cada Historia de Usuario (HU), apoyándonos en la herramienta Fork para facilitar este proceso.

Debido a dificultades iniciales con el manejo de ambientes en los diferentes repositorios, se decidió consolidar un ambiente final en el frontend, aplicando correctamente el flujo dev → qa → staging → main. En este ambiente se integraron y documentaron los cambios finales y las HU aprobadas, con el objetivo de demostrar y evidenciar el proceso de gestión de ambientes que se aprendió e implementó durante el proyecto.

Ejemplo real HU-42

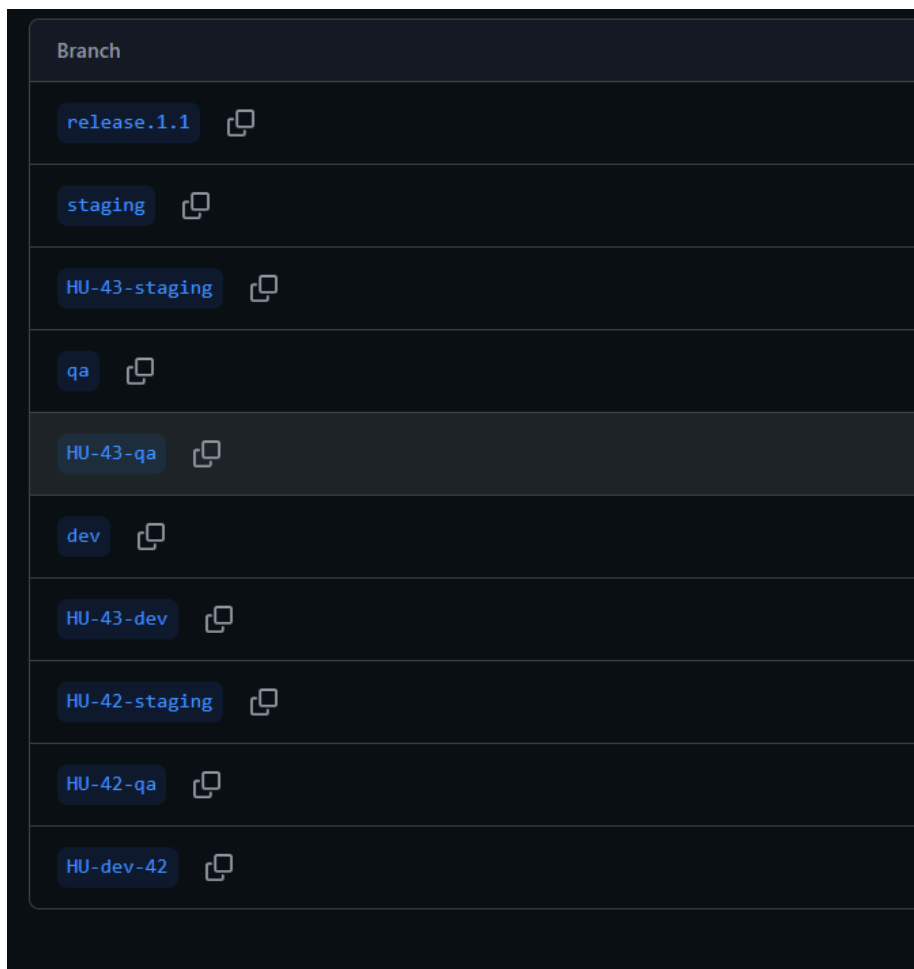
Para **HU-42** se siguió este proceso:

1. Se creó **HU-42-dev**
2. Se desarrolló la funcionalidad
3. Se hizo merge hacia **dev**
4. Se creó **HU-42-qa** y se pasó con **cherry pick**
5. Luego se pasó a **HU-42-staging**
6. Se integró con staging
7. Se incluyó en el release final

Ejemplo real HU-43

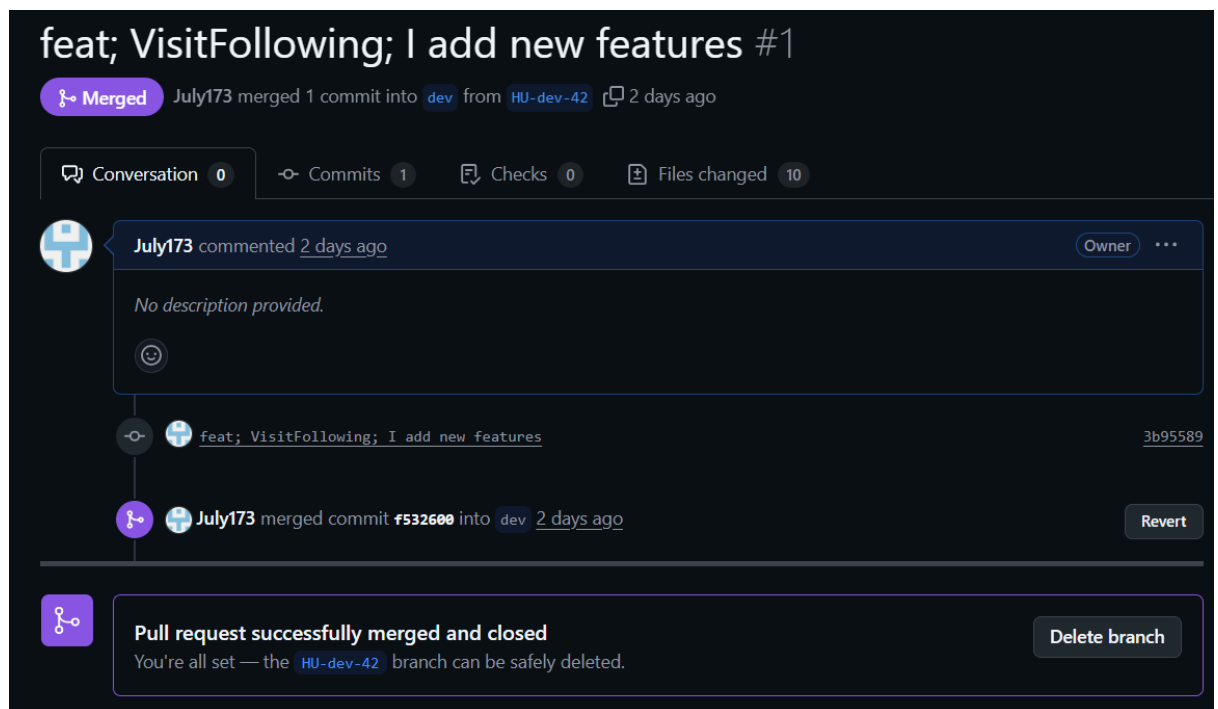
La **HU-43** repitió el proceso:

- HU-43-dev → HU-43-qa → HU-43-staging
- Se aplicaron merge request entre ambientes
- Finalmente pasó al release
- Y del release se integró a main

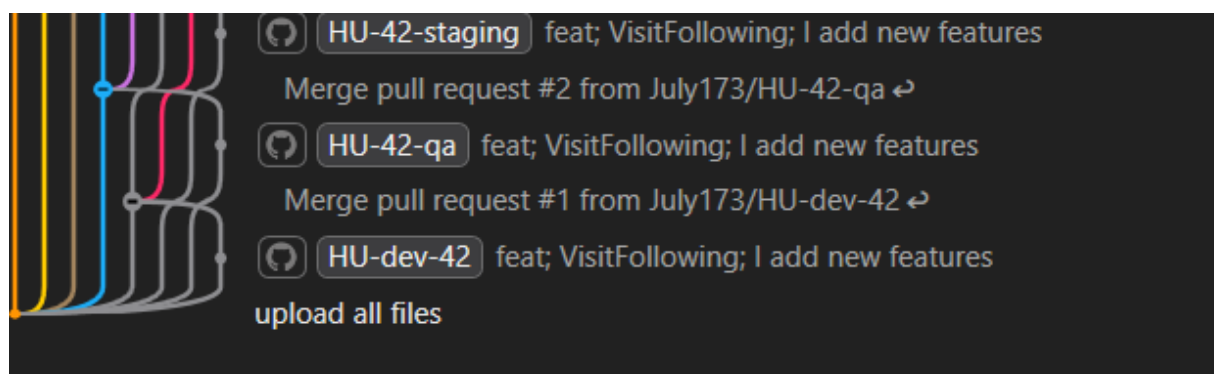


dev:

Se creo la HU-42-dev se hizo el proceso y se hizo el merge a la dev:



Posteriormente, los cambios desarrollados en la rama **HU-42-dev** fueron promovidos hacia la rama **HU-42-qa**, y luego hacia **HU-42-staging**, siguiendo el flujo de ambientes establecido. Durante este proceso se realizaron los **merge correspondientes entre cada HU y su rama principal**, es decir, primero desde la HU hacia **qa**, y luego desde la HU hacia **staging**, garantizando que cada nivel recibiera únicamente los cambios aprobados y validados para ese ambiente.



feat; VisitFollowing; I add new features #2

Merged July173 merged 1 commit into `qa` from `HU-42-qa` 2 days ago

Conversation 0 Commits 1 Checks 0 Files changed 10

July173 commented 2 days ago Owner ...

No description provided.

feat; VisitFollowing; I add new features 20d3d95

July173 merged commit 4bca483 into `qa` 2 days ago Revert

Pull request successfully merged and closed Delete branch

You're all set — the `HU-42-qa` branch can be safely deleted.

feat; VisitFollowing; I add new features #3

Merged July173 merged 1 commit into `staging` from `HU-42-staging` 2 days ago

Conversation 0 Commits 1 Checks 0 Files changed 10

July173 commented 2 days ago Owner ...

No description provided.

feat; VisitFollowing; I add new features a0cd0b4

July173 merged commit bfca153 into `staging` 2 days ago Revert

Pull request successfully merged and closed Delete branch

You're all set — the `HU-42-staging` branch can be safely deleted.

Posteriormente, se aplicó el mismo flujo de trabajo con la HU-43, trasladando los cambios desde la rama **HU-43-dev** hacia **HU-43-staging**, siguiendo las etapas correspondientes.

Hu 43 dev #4

 Merged

July173 merged 2 commits into `dev` from `HU-43-dev` 2 days ago



Conversation 0



Commits 2



Checks 0



Files changed 23



July173 commented 2 days ago

Owner ...

No description provided.



July173 added 2 commits 2 days ago



feat; Following; I added new features for the page about following.

1075cf4



fix; I fixed some errors;

295f4c9



July173 merged commit `7baa29a` into `dev` 2 days ago

Revert

Hu 43 qa #5

 Merged

July173 merged 2 commits into `qa` from `HU-43-qa` 2 days ago



Conversation 0



Commits 2



Checks 0



Files changed 23



July173 commented 2 days ago

Owner ...

No description provided.



July173 added 2 commits 2 days ago



feat; Following; I added new features for the page about following.

9477b39



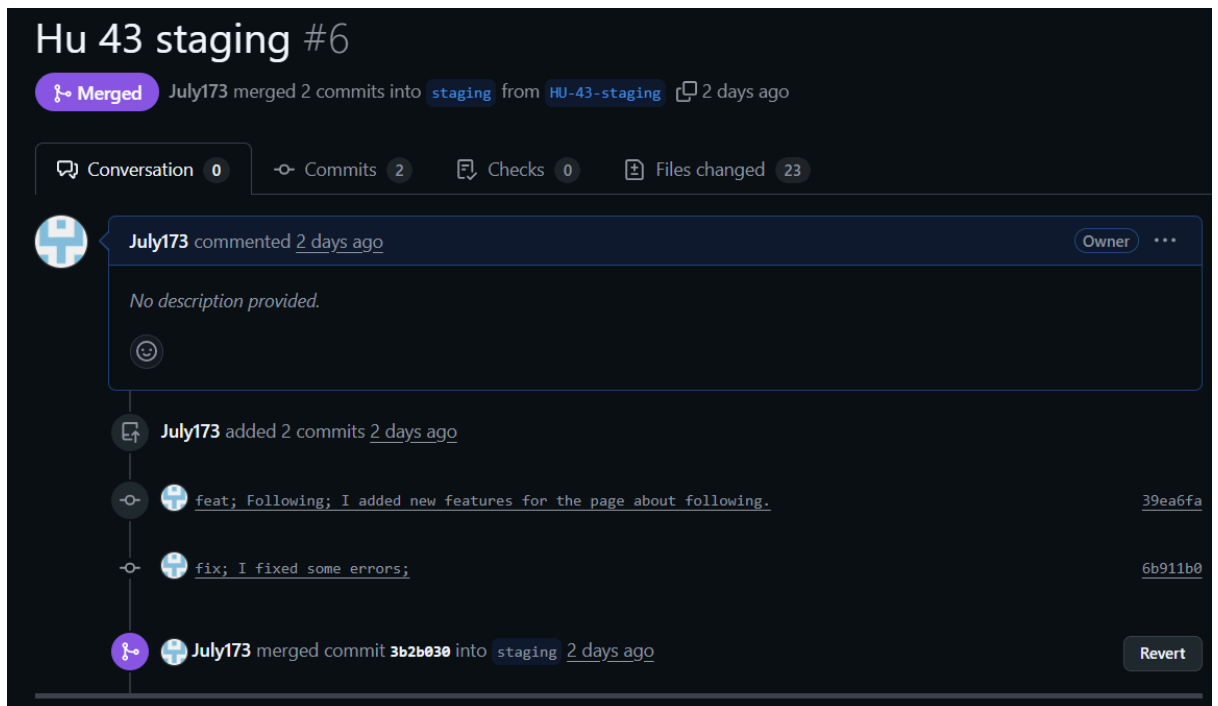
fix; I fixed some errors;

759bd4d

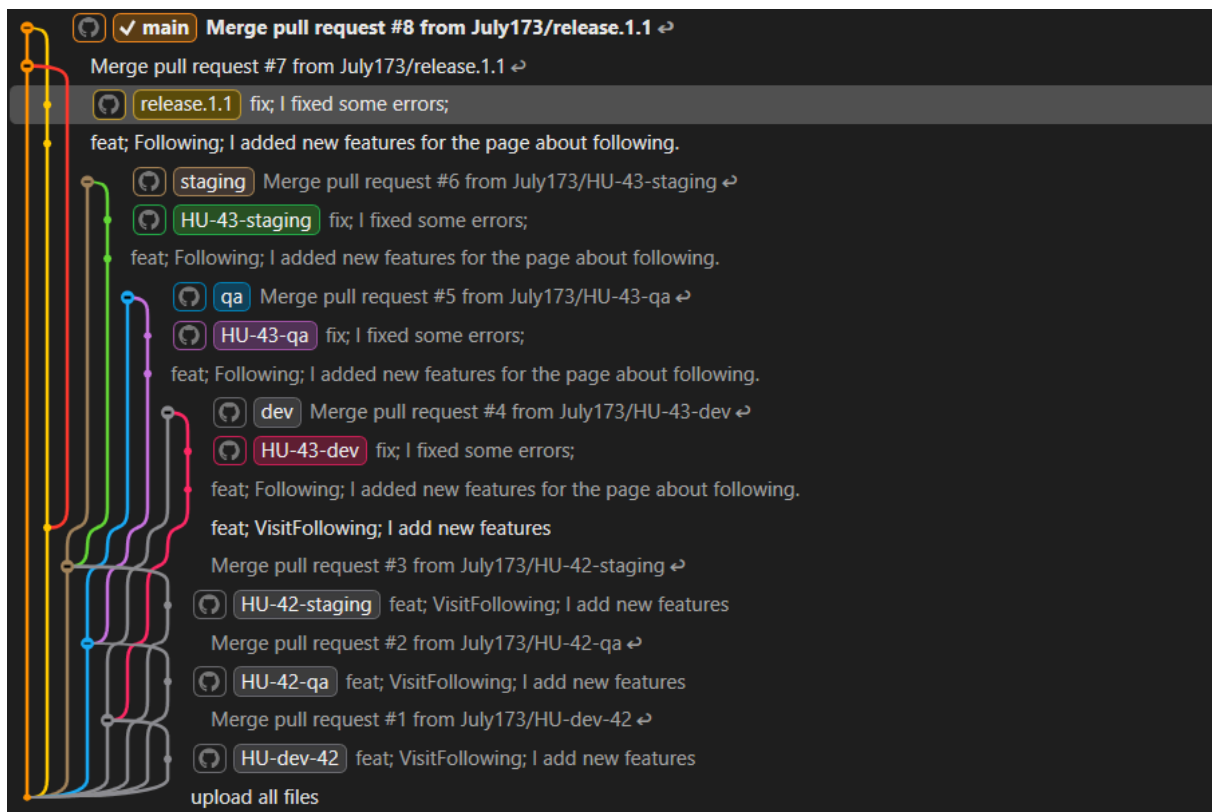


July173 merged commit `801d59a` into `qa` 2 days ago

Revert



Una vez validadas las historias de usuario en los entornos previos, se procedió a generar los **release**, consolidando las funcionalidades aprobadas. Finalmente, estos cambios fueron integrados en la rama **main**, completando el ciclo de publicación.



Release.1.1 #8

Merged July173 merged 2 commits into `main` from `release.1.1` 1 hour ago

Conversation 0 Commits 2 Checks 0 Files changed 23



July173 commented 1 hour ago Owner ...

No description provided.



July173 added 2 commits yesterday

feat; Following; I added new features for the page about following. `f11d3bc`

fix; I fixed some errors; `f57ce65`

July173 merged commit `631dedb` into `main` 1 hour ago Revert

10. Uso de Cherry Pick y Fork

En este proyecto, los **cherry-pick** se utilizaron para trasladar commits específicos de una HU a otra HU cuando era necesario compartir cambios o corregir problemas entre ramas que **no podían fusionarse por merge directo**.

Esto se debe a que:

- Las HU son ramas independientes que no deben mezclarse directamente.
- No se pueden hacer merges entre ramas HU-xx-dev o entre HU-xx-qa.
- Para mover un commit desde una HU a otra se debe hacer:

```
git cherry-pick <id_commit>
```

Los cherry-pick se realizaron en múltiples casos, sobre todo durante:

- El proceso de HU-42 (dev → qa → staging).
- El proceso posterior de HU-43.
- La integración final en los releases.

9. Conclusiones

- El manejo de ambientes permitió un flujo organizado y controlado.
- El **frontend** fue el repositorio con implementación correcta y completa.
- El **backend y móvil** presentaron fallos, pero lograron mantener un flujo funcional.
- Este esquema de ambientes facilita revisiones, pruebas y despliegues seguros.
- Es una práctica recomendada para futuros desarrollos del Sistema Autogestión SENA.